

Material de Estudo

Biblioteca API

Guia completo para entender, explicar, pesquisar e replicar a API REST de gerenciamento de biblioteca feita em Java, Spring Boot, PostgreSQL, Docker, JPA, DTOs, validações, Swagger e testes.

Gerado em 25/06/2026 22:21

Projeto: biblioteca-api

Autor do repositório: camilolucass

Sumario

1. Visao geral do projeto
2. Conceitos fundamentais
3. Arquitetura em camadas
4. Fluxos principais
5. Banco de dados e conexoes
6. Endpoints REST
7. Como rodar, testar e parar
8. Como estudar e replicar
- 9.Codigo completo comentado por arquivo

1. Visao Geral Do Projeto

A **Biblioteca API** e uma API REST backend para gerenciar livros e empréstimos. Ela permite cadastrar livros, consultar disponibilidade, realizar empréstimos e registrar devoluções.

Resultado pratico: o projeto mostra habilidades de backend reais: Java, Spring Boot, API REST, banco relacional, JPA/Hibernate, validacao, tratamento de erros, Docker, Swagger, testes unitarios e Git/GitHub.

Principais funcionalidades

- Cadastrar, listar, buscar, atualizar e remover livros.
- Consultar se um livro esta disponivel para emprestimo.
- Realizar emprestimo de livro disponivel.
- Registrar devolucao de livro emprestado.
- Listar emprestimos ativos e atrasados.
- Bloquear ISBN duplicado.
- Bloquear exclusao de livro emprestado.
- Responder erros em formato padronizado.

Tecnologias usadas

Tecnologia	Uso no projeto	O que pesquisar
Java 17+	Linguagem principal.	POO, record, enum, Optional, LocalDate.
Spring Boot	Framework para criar a API rapidamente.	@SpringBootApplication, IoC, auto configuration.
Spring Web	Controllers e endpoints HTTP.	@RestController, @GetMapping, @PostMapping.
Spring Data JPA	Acesso ao banco com repositories.	JpaRepository, query methods, @Query.
Hibernate	Implementacao JPA que mapeia objetos para tabelas.	ORM, Entity, relacionamento ManyToOne.
Bean Validation	Validacao dos DTOs de entrada.	@NotBlank, @NotNull, @Email, @Valid.
PostgreSQL	Banco principal do projeto.	Banco relacional, tabelas, chaves estrangeiras.
Docker Compose	Sobe o PostgreSQL localmente.	containers, volumes, portas, services.
H2	Banco opcional em memoria para rodar sem Docker.	profiles do Spring, banco em memoria.
Swagger/OpenAPI	Documentacao interativa da API.	springdoc-openapi, swagger-ui.
JUnit e Mockito	Testes unitarios das regras de negocio.	@Mock, @InjectMocks, asserts.

2. Conceitos Fundamentais

API REST

Uma API REST permite que outros clientes, como navegador, Postman, frontend ou app mobile, conversem com o backend usando HTTP. Cada rota representa uma ação ou recurso.

- **GET:** buscar informações.
- **POST:** criar um recurso novo.
- **PUT:** atualizar ou executar uma alteração importante.
- **DELETE:** remover um recurso.

JSON

JSON é o formato usado para enviar e receber dados na API. Exemplo de livro:

```
{
  "titulo": "Clean Code",
  "autor": "Robert C. Martin",
  "isbn": "9780132350884",
  "anoPublicacao": 2008,
  "categoria": "Programacao"
}
```

JPA, Hibernate e ORM

JPA é uma especificação Java para persistir objetos em banco de dados. Hibernate é a implementação usada pelo Spring Boot. ORM significa Object-Relational Mapping: classes Java viram tabelas, atributos viram colunas e objetos viram registros.

DTO

DTO significa Data Transfer Object. Ele separa o modelo interno do banco da entrada e saída da API. Isso evita expor a entidade JPA diretamente e permite controlar melhor validações e respostas.

Service Layer

A camada service guarda as regras de negócio. Controller não deve decidir se um livro pode ser emprestado; quem decide isso é o service.

Repository

Repository conversa com o banco. Ao estender `JpaRepository`, o Spring já entrega métodos como `save`, `findById`, `findAll` e `delete`.

Injeção de Dependência

O Spring cria e injeta objetos automaticamente. Por exemplo, o controller recebe o service no construtor; o service recebe repositories. Isso reduz acoplamento e facilita testes.

3. Arquitetura Em Camadas

O projeto foi dividido em pacotes com responsabilidades claras.



Pacote	Responsabilidade	Exemplo
controller	Receber requisicoes HTTP e devolver respostas.	LivroController
service	Aplicar regras de negocio.	EmprestimoService
repository	Acessar o banco de dados.	LivroRepository
model	Representar tabelas JPA.	Livro, Emprestimo
dto	Representar entrada e saida da API.	LivroRequestDTO
enums	Guardar valores fixos de status.	StatusLivro
exception	Padronizar erros.	GlobalExceptionHandler

Regra de ouro: controller nao deve conter regra de negocio; repository nao deve conter regra de negocio; entidade nao deve ser exposta diretamente como contrato da API. O service coordena a regra.

4. Fluxos Principais

Fluxo de cadastro de livro

1. Cliente envia POST /api/livros com JSON.
2. LivroController recebe LivroRequestDTO.
3. Bean Validation valida titulo, autor e ISBN.
4. LivroService verifica ano de publicacao e ISBN duplicado.
5. Service cria entidade Livro com status DISPONIVEL.
6. LivroRepository salva no PostgreSQL.
7. API retorna LivroResponseDTO.

Fluxo de emprestimo

1. Cliente envia POST /api/emprestimos com livroId, nome e email.
2. EmprestimoService busca o livro.
3. Se o livro nao existir, retorna 404.
4. Se o livro nao estiver DISPONIVEL, retorna erro de regra.
5. Cria emprestimo com status ATIVO.
6. Define devolucao prevista para 7 dias depois.
7. Muda o livro para EMPRESTADO.
8. Salva o emprestimo.

Fluxo de devolucao

1. Cliente envia PUT /api/emprestimos/{id}/devolucao.
2. Service busca o emprestimo.
3. Se ja estiver finalizado, bloqueia.
4. Confere se o livro esta emprestado.
5. Preenche dataDevolucao com a data atual.
6. Muda o emprestimo para FINALIZADO.
7. Muda o livro para DISPONIVEL.

5. Banco De Dados E Conexoes

PostgreSQL via Docker

O arquivo `docker-compose.yml` cria um container PostgreSQL com os dados abaixo:

Item	Valor
Banco	<code>biblioteca_db</code>
Usuario	<code>biblioteca_user</code>
Senha	<code>biblioteca_pass</code>
Porta local	<code>5432</code>
URL JDBC	<code>jdbc:postgresql://localhost:5432/biblioteca_db</code>

Conexao da aplicacao

A aplicacao usa o arquivo `application.properties` para conectar no PostgreSQL. A propriedade `spring.jpa.hibernate.ddl-auto=update` permite que o Hibernate crie/atualize as tabelas durante o desenvolvimento.

H2 opcional

O perfil `local` usa H2 em memoria. Ele serve para estudar e rodar rapido sem Docker, mas o caminho principal do projeto continua sendo PostgreSQL.

Importante: banco H2 em memoria perde os dados ao parar a aplicacao. O PostgreSQL com volume Docker preserva dados enquanto o volume existir.

6. Endpoints REST

Livros

Metodo	Rota	Uso
POST	<code>/api/livros</code>	Cadastrar livro.
GET	<code>/api/livros</code>	Listar todos os livros.
GET	<code>/api/livros/{id}</code>	Buscar livro por ID.
GET	<code>/api/livros/{id}/disponibilidade</code>	Consultar disponibilidade.
PUT	<code>/api/livros/{id}</code>	Atualizar livro.
DELETE	<code>/api/livros/{id}</code>	Remover livro, se nao estiver emprestado.

Emprestimos

Metodo	Rota	Uso
POST	/api/emprestimos	Realizar emprestimo.
GET	/api/emprestimos	Listar emprestimos.
GET	/api/emprestimos/{id}	Buscar emprestimo por ID.
GET	/api/emprestimos/ativos	Listar emprestimos ativos.
GET	/api/emprestimos/atrasados	Listar emprestimos atrasados.
PUT	/api/emprestimos/{id}/devolucao	Registrar devolucao.

7. Como Rodar, Testar E Parar

Rodar com Docker e PostgreSQL

```
cd "C:\Users\Lucas\Documents\New project\biblioteca-api"
docker compose up -d
.\mvnw.cmd spring-boot:run
```

Acessar Swagger

```
http://localhost:8080/swagger-ui.html
```

Parar tudo

```
cd "C:\Users\Lucas\Documents\New project\biblioteca-api"
docker compose down
```

Rodar testes

```
.\mvnw.cmd test
```

Rodar sem Docker, com H2

```
.\mvnw.cmd spring-boot:run -Dspring-boot.run.profiles=local
```

Console H2

```
URL: http://localhost:8080/h2-console
JDBC URL: jdbc:h2:mem:biblioteca_db
User: sa
Password: deixe em branco
```

8. Como Estudar E Replicar

Ordem recomendada de estudo

1. Java basico: classes, objetos, metodos, construtores, enums, records.
2. HTTP e REST: metodos, status codes, JSON.
3. Spring Boot: controllers, services, repositories e injeção de dependencia.
4. JPA/Hibernate: entity, id, coluna, relacionamento, repository.
5. Banco relacional: tabelas, chaves primarias, chaves estrangeiras.
6. DTO e validacao: separar entrada/saida da entidade.
7. Tratamento de erros: exceptions e RestControllerAdvice.
8. Docker: container, imagem, volume, porta, compose.
9. Testes: JUnit, Mockito, mock, assert.
10. Git/GitHub: init, add, commit, push e README.

Ordem para refazer o projeto do zero

1. Criar projeto no Spring Initializr.
2. Adicionar dependencias Web, JPA, Validation, PostgreSQL, Lombok, Swagger e Test.
3. Criar entidades Livro e Emprestimo.
4. Criar enums de status.
5. Criar repositories.
6. Criar DTOs de request e response.
7. Criar services com regras de negocio.
8. Criar controllers com endpoints.
9. Criar tratamento global de erros.
10. Criar docker-compose.
11. Criar testes unitarios.
12. Rodar testes e testar no Swagger.
13. Escrever README e publicar no GitHub.

Termos para pesquisar

Spring Boot REST API, DTO Java record, Spring Data JPA query methods, Hibernate Entity Mapping, Bean Validation, ControllerAdvice, JUnit 5 Mockito, Docker Compose PostgreSQL, Swagger OpenAPI Springdoc.

9. Código Completo Comentado Por Arquivo

Esta parte contém os arquivos principais do projeto. Leia primeiro a explicação acima; depois use estes códigos como referência para entender como cada camada conversa com a outra.

Maven e dependências

pom.xml

Define o projeto Maven, a versão do Java, Spring Boot e bibliotecas usadas: Web, JPA, Validation, PostgreSQL, H2, Lombok, Swagger e testes.

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.5.3</version>
        <relativePath/> <!-- lookup parent from repository -->
    </parent>
    <groupId>com.lucas</groupId>
    <artifactId>biblioteca-api</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>biblioteca-api</name>
    <description>API REST para gerenciamento de biblioteca</description>
    <url/>
    <licenses>
        <license/>
    </licenses>
    <developers>
        <developer/>
    </developers>
    <scm>
        <connection/>
        <developerConnection/>
        <tag/>
        <url/>
    </scm>
    <properties>
        <java.version>17</java.version>
    </properties>
    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-validation</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
        <dependency>
```

```

        <groupId>org.springdoc</groupId>
        <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
        <version>2.8.6</version>
    </dependency>

    <dependency>
        <groupId>org.postgresql</groupId>
        <artifactId>postgresql</artifactId>
        <scope>runtime</scope>
    </dependency>
    <dependency>
        <groupId>com.h2database</groupId>
        <artifactId>h2</artifactId>
        <scope>runtime</scope>
    </dependency>
    <dependency>
        <groupId>org.projectlombok</groupId>
        <artifactId>lombok</artifactId>
        <optional>true</optional>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
    </dependency>
</dependencies>

<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
            <configuration>
                <excludes>
                    <exclude>
                        <groupId>org.projectlombok</groupId>
                        <artifactId>lombok</artifactId>
                    </exclude>
                </excludes>
            </configuration>
        </plugin>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-compiler-plugin</artifactId>
            <executions>
                <execution>
                    <id>default-compile</id>
                    <phase>compile</phase>
                    <goals>
                        <goal>compile</goal>
                    </goals>
                    <configuration>
                        <annotationProcessorPaths>
                            <path>
                                <groupId>org.projectlombok</groupId>
                                <artifactId>lombok</artifactId>
                            </path>
                        </annotationProcessorPaths>

```

```

                </configuration>
            </execution>
            <execution>
                <id>default-testCompile</id>
                <phase>test-compile</phase>
                <goals>
                    <goal>testCompile</goal>
                </goals>
                <configuration>
                    <annotationProcessorPaths>
                        <path>
                            <groupId>org.projectlombok</groupId>
                            <artifactId>lombok</artifactId>
                        </path>
                    </annotationProcessorPaths>
                </configuration>
            </execution>
        </executions>
    </plugin>
</plugins>
</build>
</project>

```

Docker Compose do PostgreSQL

docker-compose.yml

Cria um container PostgreSQL local com banco, usuario, senha, porta e volume persistente.

```

services:
  postgres:
    image: postgres:16-alpine
    container_name: biblioteca_postgres
    environment:
      POSTGRES_DB: biblioteca_db
      POSTGRES_USER: biblioteca_user
      POSTGRES_PASSWORD: biblioteca_pass
    ports:
      - "5432:5432"
    volumes:
      - biblioteca_postgres_data:/var/lib/postgresql/data

volumes:
  biblioteca_postgres_data:

```

Configuracao principal com PostgreSQL

src/main/resources/application.properties

Configuracao padrao da API para conectar no PostgreSQL que sobe pelo Docker Compose.

```

spring.application.name=biblioteca-api

spring.datasource.url=jdbc:postgresql://localhost:5432/biblioteca_db
spring.datasource.username=biblioteca_user
spring.datasource.password=biblioteca_pass

```

```
spring.datasource.driver-class-name=org.postgresql.Driver

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

springdoc.swagger-ui.path=/swagger-ui.html
springdoc.api-docs.path=/v3/api-docs
```

Perfil local com H2

src/main/resources/application-local.properties

Configuracao alternativa para rodar sem Docker, usando banco H2 em memoria.

```
spring.datasource.url=jdbc:h2:mem:biblioteca_db;DB_CLOSE_DELAY=-1;DB_CLOSE_ON_EXIT=FALSE
spring.datasource.username=sa
spring.datasource.password=
spring.datasource.driver-class-name=org.h2.Driver

spring.jpa.hibernate.ddl-auto=create-drop
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

spring.h2.console.enabled=true
spring.h2.console.path=/h2-console
```

Classe principal Spring Boot

src/main/java/com/lucas/bibliotecaapi/BibliotecaApiApplication.java

Ponto de entrada da aplicacao. A anotacao @SpringBootApplication habilita a configuracao automatica e o scan de componentes.

```
package com.lucas.bibliotecaapi;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class BibliotecaApiApplication {

    public static void main(String[] args) {
        SpringApplication.run(BibliotecaApiApplication.class, args);
    }

}
```

Enum StatusLivre

src/main/java/com/lucas/bibliotecaapi/enums/StatusLivre.java

Lista os estados possiveis de um livro no sistema.

```
package com.lucas.bibliotecaapi.enums;

public enum StatusLivre {
```

```
DISPONIVEL,  
EMPRESTADO,  
INATIVO  
}
```

Enum StatusEmprestimo

src/main/java/com/lucas/bibliotecaapi/enums/StatusEmprestimo.java

Lista os estados possíveis de um empréstimo.

```
package com.lucas.bibliotecaapi.enums;  
  
public enum StatusEmprestimo {  
    ATIVO,  
    FINALIZADO,  
    ATRASADO  
}
```

Entidade Livro

src/main/java/com/lucas/bibliotecaapi/model/Livro.java

Representa a tabela livros. Usa JPA para mapear campos Java para colunas no banco.

```
package com.lucas.bibliotecaapi.model;  
  
import com.lucas.bibliotecaapi.enums.StatusLivro;  
import jakarta.persistence.Column;  
import jakarta.persistence.Entity;  
import jakarta.persistence.EnumType;  
import jakarta.persistence.Enumerated;  
import jakarta.persistence.GeneratedValue;  
import jakarta.persistence.GenerationType;  
import jakarta.persistence.Id;  
import jakarta.persistence.PrePersist;  
import jakarta.persistence.Table;  
import java.time.LocalDateTime;  
import lombok.AllArgsConstructor;  
import lombok.Builder;  
import lombok.Getter;  
import lombok.NoArgsConstructor;  
import lombok.Setter;  
  
@Entity  
@Table(name = "livros")  
@Getter  
@Setter  
@Builder  
@NoArgsConstructor  
@AllArgsConstructor  
public class Livro {  
  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private Long id;  
  
    @Column(nullable = false, length = 150)
```

```

private String titulo;

@Column(nullable = false, length = 100)
private String autor;

@Column(nullable = false, unique = true, length = 30)
private String isbn;

@Column(name = "ano_publicacao")
private Integer anoPublicacao;

@Column(length = 80)
private String categoria;

@Enumerated(EnumType.STRING)
@Column(nullable = false, length = 20)
private StatusLivro status;

@Column(name = "data_cadastro", nullable = false, updatable = false)
private LocalDateTime dataCadastro;

@PrePersist
public void prePersist() {
    if (status == null) {
        status = StatusLivro.DISPONIVEL;
    }
    if (dataCadastro == null) {
        dataCadastro = LocalDateTime.now();
    }
}
}

```

Entidade Emprestimo

src/main/java/com/lucas/bibliotecaapi/model/Emprestimo.java

Representa a tabela empréstimos. O @ManyToOne cria a relação entre empréstimo e livro.

```

package com.lucas.bibliotecaapi.model;

import com.lucas.bibliotecaapi.enums.StatusEmprestimo;
import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.EnumType;
import jakarta.persistence.Enumerated;
import jakarta.persistence.FetchType;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.ManyToOne;
import jakarta.persistence.Table;
import java.time.LocalDate;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

```

```

@Entity
@Table(name = "emprestimos")
@Getter
@Setter
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class Emprestimo {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "livro_id", nullable = false)
    private Livro livro;

    @Column(name = "nome_usuario", nullable = false, length = 100)
    private String nomeUsuario;

    @Column(name = "email_usuario", length = 120)
    private String emailUsuario;

    @Column(name = "data_emprestimo", nullable = false)
    private LocalDate dataEmprestimo;

    @Column(name = "data_prevista_devolucao", nullable = false)
    private LocalDate dataPrevistaDevolucao;

    @Column(name = "data_devolucao")
    private LocalDate dataDevolucao;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 20)
    private StatusEmprestimo status;
}

```

DTO LivroRequestDTO

src/main/java/com/lucas/bibliotecaapi/dto/LivroRequestDTO.java

DTO de entrada para cadastrar ou atualizar livro. Também contém validações de campos obrigatórios.

```

package com.lucas.bibliotecaapi.dto;

import jakarta.validation.constraints.NotBlank;

public record LivroRequestDTO(
    @NotBlank(message = "Título é obrigatório.")
    String titulo,

    @NotBlank(message = "Autor é obrigatório.")
    String autor,

    @NotBlank(message = "ISBN é obrigatório.")
    String isbn,

    Integer anoPublicacao,

```

```
        String categoria
    ) {
    }
```

DTO LivroResponseDTO

src/main/java/com/lucas/bibliotecaapi/dto/LivroResponseDTO.java

DTO de saída para devolver livro ao cliente da API sem expor diretamente a entidade JPA.

```
package com.lucas.bibliotecaapi.dto;

import com.lucas.bibliotecaapi.enums.StatusLivro;
import java.time.LocalDateTime;

public record LivroResponseDTO(
    Long id,
    String titulo,
    String autor,
    String isbn,
    Integer anoPublicacao,
    String categoria,
    StatusLivro status,
    LocalDateTime dataCadastro
) {
}
```

DTO DisponibilidadeLivroDTO

src/main/java/com/lucas/bibliotecaapi/dto/DisponibilidadeLivroDTO.java

DTO específico para a consulta de disponibilidade do livro.

```
package com.lucas.bibliotecaapi.dto;

import com.lucas.bibliotecaapi.enums.StatusLivro;

public record DisponibilidadeLivroDTO(
    Long livroId,
    String titulo,
    boolean disponivel,
    StatusLivro status
) {
}
```

DTO EmprestimoRequestDTO

src/main/java/com/lucas/bibliotecaapi/dto/EmprestimoRequestDTO.java

DTO de entrada para realizar um empréstimo.

```
package com.lucas.bibliotecaapi.dto;

import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;

public record EmprestimoRequestDTO(
```



```
@NotNull(message = "ID do livro e obrigatorio.")
Long livroId,

@NotBlank(message = "Nome do usuario e obrigatorio.")
String nomeUsuario,

@email(message = "E-mail do usuario deve ser valido.")
String emailUsuario

) {
}
```

DTO EmprestimoResponseDTO

src/main/java/com/lucas/bibliotecaapi/dto/EmprestimoResponseDTO.java

DTO de saída com dados do empréstimo, livro, datas e status.

```
package com.lucas.bibliotecaapi.dto;

import com.lucas.bibliotecaapi.enums.StatusEmprestimo;
import java.time.LocalDate;

public record EmprestimoResponseDTO(
    Long id,
    Long livroId,
    String livro,
    String nomeUsuario,
    String emailUsuario,
    LocalDate dataEmprestimo,
    LocalDate dataPrevistaDevolucao,
    LocalDate dataDevolucao,
    StatusEmprestimo status
) {
}
```

Repository de Livro

src/main/java/com/lucas/bibliotecaapi/repository/LivroRepository.java

Interface Spring Data JPA que fornece CRUD e consultas por ISBN.

```
package com.lucas.bibliotecaapi.repository;

import com.lucas.bibliotecaapi.model.Livro;
import java.util.Optional;
import org.springframework.data.jpa.repository.JpaRepository;

public interface LivroRepository extends JpaRepository<Livro, Long> {
    Optional<Livro> findByIsbn(String isbn);

    boolean existsByIsbn(String isbn);

    boolean existsByIsbnAndIdNot(String isbn, Long id);
}
```

Repository de Emprestimo

src/main/java/com/lucas/bibliotecaapi/repository/EmprestimoRepository.java

Interface Spring Data JPA que busca empréstimos por status e atrasados.

```
package com.lucas.bibliotecaapi.repository;

import com.lucas.bibliotecaapi.enums.StatusEmprestimo;
import com.lucas.bibliotecaapi.model.Emprestimo;
import java.time.LocalDate;
import java.util.List;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;

public interface EmprestimoRepository extends JpaRepository<Emprestimo, Long> {
    List<Emprestimo> findByStatus(StatusEmprestimo status);

    @Query("""
        select e from Emprestimo e
        where e.status = com.lucas.bibliotecaapi.enums.StatusEmprestimo.ATIVO
        and e.dataPrevistaDevolucao < :dataAtual
        """)
    List<Emprestimo> findAtrasados(@Param("dataAtual") LocalDate dataAtual);
}
```

Service de Livro

src/main/java/com/lucas/bibliotecaapi/service/LivroService.java

Camada onde ficam as regras de negocio de livros: ISBN unico, ano valido, disponibilidade e remocao.

```
package com.lucas.bibliotecaapi.service;

import com.lucas.bibliotecaapi.dto.DisponibilidadeLivroDTO;
import com.lucas.bibliotecaapi.dto.LivroRequestDTO;
import com.lucas.bibliotecaapi.dto.LivroResponseDTO;
import com.lucas.bibliotecaapi.enums.StatusLivro;
import com.lucas.bibliotecaapi.exception.RecursoNaoEncontradoException;
import com.lucas.bibliotecaapi.exception.RegraNegocioException;
import com.lucas.bibliotecaapi.model.Livro;
import com.lucas.bibliotecaapi.repository.LivroRepository;
import java.time.LocalDateTime;
import java.time.Year;
import java.util.List;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
@RequiredArgsConstructor
public class LivroService {

    private final LivroRepository livroRepository;

    @Transactional
    public LivroResponseDTO cadastrar(LivroRequestDTO request) {
        validarAnoPublicacao(request.anoPublicacao());
        validarIsbnDisponivel(request.isbn());

        Livro livro = Livro.builder()
```

```
        .titulo(request.titulo().trim())
        .autor(request.autor().trim())
        .isbn(request.isbn().trim())
        .anoPublicacao(request.anoPublicacao())
        .categoria(trimOrNull(request.categoria()))
        .status(StatusLivro.DISPONIVEL)
        .dataCadastro(LocalDateTime.now())
        .build();

    return toResponse(livroRepository.save(livro));
}

@Transactional(readOnly = true)
public List<LivroResponseDTO> listarTodos() {
    return livroRepository.findAll()
        .stream()
        .map(this::toResponse)
        .toList();
}

@Transactional(readOnly = true)
public LivroResponseDTO buscarPorId(Long id) {
    return toResponse(buscarLivro(id));
}

@Transactional(readOnly = true)
public DisponibilidadeLivroDTO consultarDisponibilidade(Long id) {
    Livro livro = buscarLivro(id);
    boolean disponivel = livro.getStatus() == StatusLivro.DISPONIVEL;
    return new DisponibilidadeLivroDTO(livro.getId(), livro.getTitulo(), disponivel, livro.getStatus());
}

@Transactional
public LivroResponseDTO atualizar(Long id, LivroRequestDTO request) {
    Livro livro = buscarLivro(id);
    validarAnoPublicacao(request.anoPublicacao());

    String novoIsbn = request.isbn().trim();
    if (!livro.getIsbn().equals(novoIsbn) && livroRepository.existsByIsbnAndIdNot(novoIsbn, id)) {
        throw new RegraNegocioException("ISBN ja cadastrado.");
    }

    livro.setTitulo(request.titulo().trim());
    livro.setAutor(request.autor().trim());
    livro.setIsbn(novoIsbn);
    livro.setAnoPublicacao(request.anoPublicacao());
    livro.setCategoria(trimOrNull(request.categoria()));

    return toResponse(livroRepository.save(livro));
}

@Transactional
public void remover(Long id) {
    Livro livro = buscarLivro(id);
    if (livro.getStatus() == StatusLivro.EMPRESTADO) {
        throw new RegraNegocioException("Nao e possivel excluir um livro emprestado.");
    }
    livroRepository.delete(livro);
}
```

```

private Livro buscarLivro(Long id) {
    return livroRepository.findById(id)
        .orElseThrow(() -> new RecursoNaoEncontradoException("Livro nao encontrado."));
}

private void validarIsbnDisponivel(String isbn) {
    if (livroRepository.existsByIsbn(isbn.trim())) {
        throw new RegraNegocioException("ISBN ja cadastrado.");
    }
}

private void validarAnoPublicacao(Integer anoPublicacao) {
    if (anoPublicacao != null && anoPublicacao > Year.now().getValue()) {
        throw new RegraNegocioException("Ano de publicacao nao pode ser maior que o ano atual.");
    }
}

private LivroResponseDTO toResponse(Livro livro) {
    return new LivroResponseDTO(
        livro.getId(),
        livro.getTitulo(),
        livro.getAutor(),
        livro.getIsbn(),
        livro.getAnoPublicacao(),
        livro.getCategoria(),
        livro.getStatus(),
        livro.getDataCadastro()
    );
}

private String trimToNull(String value) {
    if (value == null || value.isBlank()) {
        return null;
    }
    return value.trim();
}
}

```

Service de Emprestimo

src/main/java/com/lucas/bibliotecaapi/service/EmprestimoService.java

Camada onde ficam as regras de emprestimo e devolucao. E o coracao do fluxo da biblioteca.

```

package com.lucas.bibliotecaapi.service;

import com.lucas.bibliotecaapi.dto.EmprestimoRequestDTO;
import com.lucas.bibliotecaapi.dto.EmprestimoResponseDTO;
import com.lucas.bibliotecaapi.enums.StatusEmprestimo;
import com.lucas.bibliotecaapi.enums.StatusLivro;
import com.lucas.bibliotecaapi.exception.RecursoNaoEncontradoException;
import com.lucas.bibliotecaapi.exception.RegraNegocioException;
import com.lucas.bibliotecaapi.model.Emprestimo;
import com.lucas.bibliotecaapi.model.Livro;
import com.lucas.bibliotecaapi.repository.EmprestimoRepository;
import com.lucas.bibliotecaapi.repository.LivroRepository;
import java.time.LocalDate;
import java.util.List;

```

```
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
@RequiredArgsConstructor
public class EmprestimoService {

    private static final int DIAS_PARA_DEVOLUCAO = 7;

    private final EmprestimoRepository emprestimoRepository;
    private final LivroRepository livroRepository;

    @Transactional
    public EmprestimoResponseDTO realizarEmprestimo(EmprestimoRequestDTO request) {
        Livro livro = livroRepository.findById(request.livroId())
            .orElseThrow(() -> new RecursoNaoEncontradoException("Livro nao encontrado."));

        if (livro.getStatus() != StatusLivro.DISPONIVEL) {
            throw new RegraNegocioException("Livro nao esta disponivel para emprestimo.");
        }

        LocalDate dataEmprestimo = LocalDate.now();
        Emprestimo emprestimo = Emprestimo.builder()
            .livro(livro)
            .nomeUsuario(request.nomeUsuario().trim())
            .emailUsuario(trimToNull(request.emailUsuario()))
            .dataEmprestimo(dataEmprestimo)
            .dataPrevistaDevolucao(dataEmprestimo.plusDays(DIAS_PARA_DEVOLUCAO))
            .status(StatusEmprestimo.ATIVO)
            .build();

        livro.setStatus(StatusLivro.EMPRESTADO);
        return toResponse(emprestimoRepository.save(emprestimo));
    }

    @Transactional(readOnly = true)
    public List<EmprestimoResponseDTO> listarTodos() {
        return emprestimoRepository.findAll()
            .stream()
            .map(this::toResponse)
            .toList();
    }

    @Transactional(readOnly = true)
    public EmprestimoResponseDTO buscarPorId(Long id) {
        return toResponse(buscarEmprestimo(id));
    }

    @Transactional(readOnly = true)
    public List<EmprestimoResponseDTO> listarAtivos() {
        return emprestimoRepository.findByStatus(StatusEmprestimo.ATIVO)
            .stream()
            .map(this::toResponse)
            .toList();
    }

    @Transactional(readOnly = true)
    public List<EmprestimoResponseDTO> listarAtrasados() {
```

```
        return emprestimoRepository.findAtrasados(LocalDate.now())
            .stream()
            .map(emprestimo -> toResponse(emprestimo, StatusEmprestimo.ATRASADO))
            .toList();
    }

    @Transactional
    public EmprestimoResponseDTO devolver(Long id) {
        Emprestimo emprestimo = buscarEmprestimo(id);
        StatusEmprestimo statusAtual = statusEfetivo(emprestimo);

        if (statusAtual == StatusEmprestimo.FINALIZADO) {
            throw new RegraNegocioException("Este emprestimo ja foi finalizado.");
        }

        if (statusAtual != StatusEmprestimo.ATIVO && statusAtual != StatusEmprestimo.ATRASADO) {
            throw new RegraNegocioException("Emprestimo deve estar ativo para ser devolvido.");
        }

        Livro livro = emprestimo.getLivro();
        if (livro.getStatus() != StatusLivro.EMPRESTADO) {
            throw new RegraNegocioException("Livro deve estar emprestado para ser devolvido.");
        }

        emprestimo.setDataDevolucao(LocalDate.now());
        emprestimo.setStatus(StatusEmprestimo.FINALIZADO);
        livro.setStatus(StatusLivro.DISPONIVEL);

        return toResponse(emprestimoRepository.save(emprestimo));
    }

    private Emprestimo buscarEmprestimo(Long id) {
        return emprestimoRepository.findById(id)
            .orElseThrow(() -> new RecursoNaoEncontradoException("Emprestimo nao encontrado."));
    }

    private EmprestimoResponseDTO toResponse(Emprestimo emprestimo) {
        return toResponse(emprestimo, statusEfetivo(emprestimo));
    }

    private EmprestimoResponseDTO toResponse(Emprestimo emprestimo, StatusEmprestimo status) {
        Livro livro = emprestimo.getLivro();
        return new EmprestimoResponseDTO(
            emprestimo.getId(),
            livro.getId(),
            livro.getTitulo(),
            emprestimo.getNomeUsuario(),
            emprestimo.getEmailUsuario(),
            emprestimo.getDataEmprestimo(),
            emprestimo.getDataPrevistaDevolucao(),
            emprestimo.getDataDevolucao(),
            status
        );
    }

    private StatusEmprestimo statusEfetivo(Emprestimo emprestimo) {
        if (emprestimo.getStatus() == StatusEmprestimo.ATIVO
            && emprestimo.getDataPrevistaDevolucao().isBefore(LocalDate.now())) {
            return StatusEmprestimo.ATRASADO;
        }
    }
}
```

```

    }
    return emprestimo.getStatus();
}

private String trimToNull(String value) {
    if (value == null || value.isBlank()) {
        return null;
    }
    return value.trim();
}
}
}

```

Controller de Livro

src/main/java/com/lucas/bibliotecaapi/controller/LivroController.java

Exponibiliza os endpoints REST de livros. Recebe DTOs e chama o service.

```

package com.lucas.bibliotecaapi.controller;

import com.lucas.bibliotecaapi.dto.DisponibilidadeLivroDTO;
import com.lucas.bibliotecaapi.dto.LivroRequestDTO;
import com.lucas.bibliotecaapi.dto.LivroResponseDTO;
import com.lucas.bibliotecaapi.service.LivroService;
import jakarta.validation.Valid;
import java.util.List;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api/livros")
@RequiredArgsConstructor
public class LivroController {

    private final LivroService livroService;

    @PostMapping
    public ResponseEntity<LivroResponseDTO> cadastrar(@Valid @RequestBody LivroRequestDTO request) {
        return ResponseEntity.status(HttpStatus.CREATED).body(livroService.cadastrar(request));
    }

    @GetMapping
    public ResponseEntity<List<LivroResponseDTO>> listarTodos() {
        return ResponseEntity.ok(livroService.listarTodos());
    }

    @GetMapping("/{id}")
    public ResponseEntity<LivroResponseDTO> buscarPorId(@PathVariable Long id) {
        return ResponseEntity.ok(livroService.buscarPorId(id));
    }
}

```

```

@GetMapping("/{id}/disponibilidade")
public ResponseEntity<DisponibilidadeLivroDTO> consultarDisponibilidade(@PathVariable Long id) {
    return ResponseEntity.ok(livroService.consultarDisponibilidade(id));
}

@PutMapping("/{id}")
public ResponseEntity<LivroResponseDTO> atualizar(
    @PathVariable Long id,
    @Valid @RequestBody LivroRequestDTO request
) {
    return ResponseEntity.ok(livroService.atualizar(id, request));
}

@DeleteMapping("/{id}")
public ResponseEntity<Void> remover(@PathVariable Long id) {
    livroService.remover(id);
    return ResponseEntity.noContent().build();
}
}

```

Controller de Emprestimo

src/main/java/com/lucas/bibliotecaapi/controller/EmprestimoController.java

Exponibiliza os endpoints REST de empréstimos e devoluções.

```

package com.lucas.bibliotecaapi.controller;

import com.lucas.bibliotecaapi.dto.EmprestimoRequestDTO;
import com.lucas.bibliotecaapi.dto.EmprestimoResponseDTO;
import com.lucas.bibliotecaapi.service.EmprestimoService;
import jakarta.validation.Valid;
import java.util.List;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api/emprestimos")
@RequiredArgsConstructor
public class EmprestimoController {

    private final EmprestimoService emprestimoService;

    @PostMapping
    public ResponseEntity<EmprestimoResponseDTO> realizarEmprestimo(
        @Valid @RequestBody EmprestimoRequestDTO request
    ) {
        return ResponseEntity.status(HttpStatus.CREATED).body(emprestimoService.realizarEmprestimo(request));
    }
}

```



```

@GetMapping
public ResponseEntity<List<EmprestimoResponseDTO>> listarTodos() {
    return ResponseEntity.ok(emprestimoService.listarTodos());
}

@GetMapping("/{id}")
public ResponseEntity<EmprestimoResponseDTO> buscarPorId(@PathVariable Long id) {
    return ResponseEntity.ok(emprestimoService.buscarPorId(id));
}

@GetMapping("/ativos")
public ResponseEntity<List<EmprestimoResponseDTO>> listarAtivos() {
    return ResponseEntity.ok(emprestimoService.listarAtivos());
}

@GetMapping("/atrasados")
public ResponseEntity<List<EmprestimoResponseDTO>> listarAtrasados() {
    return ResponseEntity.ok(emprestimoService.listarAtrasados());
}

@PutMapping("/{id}/devolucao")
public ResponseEntity<EmprestimoResponseDTO> devolver(@PathVariable Long id) {
    return ResponseEntity.ok(emprestimoService.devolver(id));
}
}

```

Resposta padrao de erro

src/main/java/com/lucas/bibliotecaapi/exception/ApiErrorResponse.java

Formato padrao para mensagens de erro retornadas pela API.

```

package com.lucas.bibliotecaapi.exception;

import java.time.LocalDateTime;
import java.util.Map;

public record ApiErrorResponse(
    LocalDateTime timestamp,
    int status,
    String error,
    String message,
    String path,
    Map<String, String> fields
) {
    public static ApiErrorResponse of(
        int status,
        String error,
        String message,
        String path,
        Map<String, String> fields
    ) {
        return new ApiErrorResponse(LocalDateTime.now(), status, error, message, path, fields);
    }
}

```

Erro de recurso nao encontrado

src/main/java/com/lucas/bibliotecaapi/exception/RecursoNaoEncontradoException.java

Execcao usada quando livro ou emprestimo nao existe.

```
package com.lucas.bibliotecaapi.exception;

public class RecursoNaoEncontradoException extends RuntimeException {
    public RecursoNaoEncontradoException(String message) {
        super(message);
    }
}
```

Erro de regra de negocio

src/main/java/com/lucas/bibliotecaapi/exception/RegraNegocioException.java

Execcao usada quando uma regra de negocio e violada.

```
package com.lucas.bibliotecaapi.exception;

public class RegraNegocioException extends RuntimeException {
    public RegraNegocioException(String message) {
        super(message);
    }
}
```

Tratamento global de erros

src/main/java/com/lucas/bibliotecaapi/exception/GlobalExceptionHandler.java

Centraliza a traducao de excecoes Java para respostas HTTP organizadas.

```
package com.lucas.bibliotecaapi.exception;

import jakarta.servlet.http.HttpServletRequest;
import java.util.LinkedHashMap;
import java.util.Map;
import org.springframework.dao.DataIntegrityViolationException;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler({RecursoNaoEncontradoException.class})
    public ResponseEntity<ApiErrorResponse> handleRecursoNaoEncontrado(
        RecursoNaoEncontradoException exception,
        HttpServletRequest request
    ) {
        return buildResponse(HttpStatus.NOT_FOUND, exception.getMessage(), request, null);
    }

    @ExceptionHandler({RegraNegocioException.class})
    public ResponseEntity<ApiErrorResponse> handleRegraNegocio(
```

```

        RegraNegocioException exception,
        HttpServletRequest request
    ) {
        return buildResponse(HttpStatus.BAD_REQUEST, exception.getMessage(), request, null);
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ApiErrorResponse> handleValidacao(
        MethodArgumentNotValidException exception,
        HttpServletRequest request
    ) {
        Map<String, String> fields = new LinkedHashMap<>();
        exception.getBindingResult().getFieldErrors()
            .forEach(error -> fields.put(error.getField(), error.getDefaultMessage()));

        return buildResponse(HttpStatus.BAD_REQUEST, "Dados da requisicao invalidos.", request, fields);
    }

    @ExceptionHandler(DataIntegrityViolationException.class)
    public ResponseEntity<ApiErrorResponse> handleDataIntegrity(
        DataIntegrityViolationException exception,
        HttpServletRequest request
    ) {
        return buildResponse(
            HttpStatus.CONFLICT,
            "Operacao viola uma restricao do banco de dados.",
            request,
            null
        );
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ApiErrorResponse> handleException(Exception exception, HttpServletRequest request) {
        return buildResponse(HttpStatus.INTERNAL_SERVER_ERROR, "Erro interno do servidor.", request, null);
    }

    private ResponseEntity<ApiErrorResponse> buildResponse(
        HttpStatus status,
        String message,
        HttpServletRequest request,
        Map<String, String> fields
    ) {
        ApiErrorResponse response = ApiErrorResponse.of(
            status.value(),
            status.getReasonPhrase(),
            message,
            request.getRequestURI(),
            fields
        );
        return ResponseEntity.status(status).body(response);
    }
}

```

Teste da aplicacao

src/test/java/com/lucas/bibliotecaapi/BibliotecaApiApplicationTests.java

Teste simples para manter a classe principal coberta sem exigir banco real.

```
package com.lucas.bibliotecaapi;

import static org.assertj.core.api.Assertions.assertThat;

import org.junit.jupiter.api.Test;

class BibliotecaApiApplicationTests {

    @Test
    void applicationClassExists() {
        assertThat(BibliotecaApiApplication.class).isNotNull();
    }

}
```

Testes do LivroService

src/test/java/com/lucas/bibliotecaapi/service/LivroServiceTest.java

Teste unitario da regra de ISBN duplicado.

```
package com.lucas.bibliotecaapi.service;

import static org.assertj.core.api.Assertions.assertThatThrownBy;
import static org.mockito.Mockito.verify;
import static org.mockito.Mockito.when;

import com.lucas.bibliotecaapi.dto.LivroRequestDTO;
import com.lucas.bibliotecaapi.exception.RegraNegocioException;
import com.lucas.bibliotecaapi.repository.LivroRepository;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

@ExtendWith(MockitoExtension.class)
class LivroServiceTest {

    @Mock
    private LivroRepository livroRepository;

    @InjectMocks
    private LivroService livroService;

    @Test
    void naoPermitirCadastrarIsbnDuplicado() {
        LivroRequestDTO request = new LivroRequestDTO(
            "Clean Code",
            "Robert C. Martin",
            "9780132350884",
            2008,
            "Programacao"
        );

        when(livroRepository.existsByIsbn("9780132350884")).thenReturn(true);

        assertThatThrownBy(() -> livroService.cadastrar(request))
```

```

        .assertInstanceOf(RegraNegocioException.class)
        .hasMessage("ISBN ja cadastrado.");

        verify(livroRepository).existsByIsbn("9780132350884");
    }
}

```

Testes do EmprestimoService

src/test/java/com/lucas/bibliotecaapi/service/EmprestimoServiceTest.java

Testes unitarios das principais regras de emprestimo e devolucao.

```

package com.lucas.bibliotecaapi.service;

import static java.time.temporal.ChronoUnit.DAYS;
import static org.assertj.core.api.Assertions.assertThat;
import static org.assertj.core.api.Assertions.assertThatThrownBy;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.verifyNoInteractions;
import static org.mockito.Mockito.when;

import com.lucas.bibliotecaapi.dto.EmprestimoRequestDTO;
import com.lucas.bibliotecaapi.dto.EmprestimoResponseDTO;
import com.lucas.bibliotecaapi.enums.StatusEmprestimo;
import com.lucas.bibliotecaapi.enums.StatusLivro;
import com.lucas.bibliotecaapi.exception.RegraNegocioException;
import com.lucas.bibliotecaapi.model.Emprestimo;
import com.lucas.bibliotecaapi.model.Livro;
import com.lucas.bibliotecaapi.repository.EmprestimoRepository;
import com.lucas.bibliotecaapi.repository.LivroRepository;
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.Optional;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

@ExtendWith(MockitoExtension.class)
class EmprestimoServiceTest {

    @Mock
    private EmprestimoRepository emprestimoRepository;

    @Mock
    private LivroRepository livroRepository;

    @InjectMocks
    private EmprestimoService emprestimoService;

    @Test
    void naoPermitirEmprestarLivroIndisponivel() {
        Livro livro = livroBase();
        livro.setStatus(StatusLivro.EMPRESTADO);
        EmprestimoRequestDTO request = requestEmprestimo();

        when(livroRepository.findById(1L)).thenReturn(Optional.of(livro));
    }

```

```
        assertThatThrownBy(() -> emprestimoService.realizarEmprestimo(request))
            .assertInstanceOf(RegraNegocioException.class)
            .hasMessage("Livro nao esta disponivel para emprestimo.");

        verifyNoInteractions(emprestimoRepository);
    }

    @Test
    void emprestarLivroDisponivelComSucesso() {
        Livro livro = livroBase();
        EmprestimoRequestDTO request = requestEmprestimo();

        when(livroRepository.findById(1L)).thenReturn(Optional.of(livro));
        when(emprestimoRepository.save(any(Emprestimo.class))).thenAnswer(invocation -> {
            Emprestimo emprestimo = invocation.getArgument(0);
            emprestimo.setId(1L);
            return emprestimo;
        });

        EmprestimoResponseDTO response = emprestimoService.realizarEmprestimo(request);

        assertThat(livro.getStatus()).isEqualTo(StatusLivro.EMPRESTADO);
        assertThat(response.status()).isEqualTo(StatusEmprestimo.ATIVO);
        assertThat(response.livroId()).isEqualTo(1L);
        assertThat(DAYS.between(response.dataEmprestimo(), response.dataPrevistaDevolucao())).isEqualTo(7);
    }

    @Test
    void devolverLivroComSucesso() {
        Livro livro = livroBase();
        livro.setStatus(StatusLivro.EMPRESTADO);
        Emprestimo emprestimo = emprestimoAtivo(livro);

        when(emprestimoRepository.findById(1L)).thenReturn(Optional.of(emprestimo));
        when(emprestimoRepository.save(any(Emprestimo.class))).thenAnswer(invocation -> invocation.getArgument(0));

        EmprestimoResponseDTO response = emprestimoService.devolver(1L);

        assertThat(livro.getStatus()).isEqualTo(StatusLivro.DISPONIVEL);
        assertThat(response.status()).isEqualTo(StatusEmprestimo.FINALIZADO);
        assertThat(response.dataDevolucao()).isEqualTo(LocalDate.now());
    }

    @Test
    void naoPermitirDevolverEmprestimoJaFinalizado() {
        Livro livro = livroBase();
        livro.setStatus(StatusLivro.DISPONIVEL);
        Emprestimo emprestimo = emprestimoAtivo(livro);
        emprestimo.setStatus(StatusEmprestimo.FINALIZADO);

        when(emprestimoRepository.findById(1L)).thenReturn(Optional.of(emprestimo));

        assertThatThrownBy(() -> emprestimoService.devolver(1L))
            .assertInstanceOf(RegraNegocioException.class)
            .hasMessage("Este emprestimo ja foi finalizado.");
    }

    private Livro livroBase() {
```

```

        return Livro.builder()
            .id(1L)
            .titulo("Clean Code")
            .autor("Robert C. Martin")
            .isbn("9780132350884")
            .anoPublicacao(2008)
            .categoria("Programacao")
            .status(StatusLivro.DISPONIVEL)
            .dataCadastro(LocalDateTime.now())
            .build();
    }

    private Emprestimo emprestimoAtivo(Livro livro) {
        LocalDate hoje = LocalDate.now();
        return Emprestimo.builder()
            .id(1L)
            .livro(livro)
            .nomeUsuario("Lucas Camilo")
            .emailUsuario("lucas@email.com")
            .dataEmprestimo(hoje)
            .dataPrevistaDevolucao(hoje.plusDays(7))
            .status(StatusEmprestimo.ATIVO)
            .build();
    }

    private EmprestimoRequestDTO requestEmprestimo() {
        return new EmprestimoRequestDTO(1L, "Lucas Camilo", "lucas@email.com");
    }
}

```

Gitignore

.gitignore

Evita versionar arquivos gerados, build, IDE e logs locais.

```

HELP.md
target/
*.log
.mvn/wrapper/maven-wrapper.jar
!**/src/main/**/target/
!**/src/test/**/target/

### STS ###
.appt_generated
.classpath
.factorypath
.project
.settings
.springBeans
.sts4-cache

### IntelliJ IDEA ###
.idea
*.iws
*.iml
*.ipr

### NetBeans ###

```

```
/nbproject/private/  
/nbbuild/  
/dist/  
/nbdist/  
/.nb-gradle/  
build/  
!*/src/main/**/build/  
!*/src/test/**/build/  
  
### VS Code ###  
.vscode/
```